

IBVAP — Intelligent Border Video Analytics Platform

01. Complete Project Overview & Executive Summary

1. Executive Summary

Project Name: IBVAP (Intelligent Border Video Analytics Platform)
Domain: AI-Powered Border Surveillance, Automated Intrusion Detection & Cryptographic Security Analytics
Hackathon: Smart India Hackathon (SIH 2026) — Blockchain & Cybersecurity Category

IBVAP is an end-to-end computer vision and security analytics platform designed to solve critical real-world border security challenges. It transforms raw, passive CCTV/thermal border video feeds into actionable, automated security intelligence.

2. High-Level Problem & Solution Statement

Problem Statement

Traditional border monitoring relies heavily on manual human observation of hundreds of camera feeds. Human surveillance operators suffer from fatigue, blind spots, slow response times, and an inability to maintain detailed, tamper-proof logs of security breaches. Furthermore, traditional logging systems (such as plain CSVs or central SQL databases) are vulnerable to internal operational tampering, accidental deletion, or record manipulation.

IBVAP Solution

IBVAP automates border perimeter surveillance by combining:

- **State-of-the-Art Object Detection & Tracking:** Real-time target identification (YOLO11) paired with multi-object temporal tracking (ByteTrack).
- **Geofenced Perimeter Protection:** Polygon-based restricted zone spatial analysis.
- **Automated Evidence Capture:** High-resolution digital snapshot generation triggered upon confirmed perimeter breaches.
- **Cryptographic Audit Integrity:** SHA-256 hash-chaining across all security events and a local block-linked audit ledger (`blockchain_ledger.json`) ensuring tamper evidence.
- **Command Center Dashboard:** A modern, executive Streamlit Web Command Center displaying real-time metrics, video feeds, evidence galleries, and cryptographic integrity checks.

3. Verification & Operational Status Summary

Capability	Status	Implementation Details
------------	--------	------------------------

YOLO11 Target Detection	IMPLEMENTED	<code>models/yolo11s.pt</code> / <code>models/yolo11n.pt</code> integrated via Ultralytics API
ByteTrack Target Tracking	IMPLEMENTED	Configured via <code>bytetrack_day3.yaml</code> with track buffer and score fusion
Restricted Zone Detection	IMPLEMENTED	Spatial polygon intersection flagging person entries (<code>day3_zone_events.csv</code>)
Digital Evidence Capture	IMPLEMENTED	Visual snapshots generated and stored in <code>output/evidence/</code>
Database Integration	IMPLEMENTED	Multi-tier fallback: MongoDB Atlas $\rightarrow$ Supabase $\rightarrow$ Local CSV $\rightarrow$ Seed Data
SHA-256 Audit Log	IMPLEMENTED	Hash-chained logging in <code>output/secure_audit_log.csv</code> & <code>audit_root_hash.txt</code>
Local Blockchain Ledger	DEMO / PROTOTYPE	Block-linked hash structure in <code>output/blockchain_ledger.json</code>
Distributed Consensus / P2P	NOT IMPLEMENTED	Flagged as future scope (Hyperledger / Ethereum integration)
Model Precision / FPS Benchmarks	NOT VERIFIED FROM SOURCE	Not experimentally measured in current codebase

4. Core Project Directory Structure

```

IBVAP_1/
■■■ bytetrack_day3.yaml          # ByteTrack tracker configuration hyperparameters
■■■ Day1_detection.ipynb        # Model evaluation & detection experimentation notebook
■■■ Day2_tracking.ipynb        # Tracking & zone detection evaluation notebook
■■■ data/
■   ■■■ video.mp4              # Input raw surveillance video feed
■■■ models/
■   ■■■ yolo11n.pt              # YOLO11 Nano model weights (5.6 MB)
■   ■■■ yolo11s.pt              # YOLO11 Small model weights (19.3 MB)
■■■ output/
■   ■■■ audit_root_hash.txt     # Root hash of SHA-256 audit chain
■   ■■■ blockchain_ledger.json  # Local block-linked JSON ledger
■   ■■■ day2_intrusion_detection.mp4 # Annotated output video (Day 2)
■   ■■■ day3_id_consistency.csv # Tracking ID consistency metric log
■   ■■■ day3_id_switch_results.csv # Tracking ID switch evaluation
■   ■■■ day3_zone_events.csv    # Zone transition event log (ENTER/EXIT)
■   ■■■ final_intrusion_video.mp4 # Primary surveillance video feed for dashboard
■   ■■■ final_surveillance_demo.mp4 # Full demo surveillance stream
■   ■■■ intrusion_log.csv       # Standard CSV intrusion log
■   ■■■ secure_audit_log.csv    # SHA-256 hash-chained audit log
■   ■■■ evidence/              # Evidence image repository
■       ■■■ EVT-001_track_2_frame_30.jpg
■       ■■■ EVT-002_track_7_frame_108.jpg
■       ■■■ EVT-003_track_36_frame_177.jpg
■■■ screenshots/               # Application UI screenshots
■   ■■■ intrusion_001.jpg
■   ■■■ intrusion_002.jpg
■   ■■■ intrusion_003.jpg
■■■ src/
■   ■■■ app.py                  # Full single-file Streamlit web app (1605 lines)
■   ■■■ main.py                 # Core OpenCV/YOLO execution script
■   ■■■ output/                 # Secondary output directory with fallback evidence
■■■ streamlit_app/             # Modularized Streamlit production app
■   ■■■ app.py                  # Main entry point & navigation router
■   ■■■ requirements.txt        # Python package requirements
■   ■■■ README.md               # Streamlit application documentation
■   ■■■ .streamlit/config.toml  # Dark theme UI configuration
■   ■■■ components/ui_components.py # Reusable HTML/CSS UI components
■   ■■■ pages/                  # Page-specific rendering modules
■       ■■■ command_center.py   # Executive dashboard
■       ■■■ surveillance.py      # Video player & zone visualization
■       ■■■ security_events.py   # Event filtering & details modal
■       ■■■ digital_evidence.py  # High-res evidence viewer
■       ■■■ audit_log.py         # SHA-256 audit log & verification
■       ■■■ system_page.py       # Architecture & module status
■       ■■■ utils/data_service.py # Database abstraction & data loader layer
■■■ SIH_PREPARATION/           # SIH Hackathon comprehensive documentation package

```